INSUREHUB

Insurance Policy Management System

Team Project Architecture & Implementation Guide

Assigned Project Topic

Insurance Policy Management API — Create, Renew, and Cancel Insurance Policies.

The goal is to build a complete web application, not only a CRUD application.

Suggested Stack: React + Spring Boot + Spring Security/JWT + JPA/Hibernate + PostgreSQL

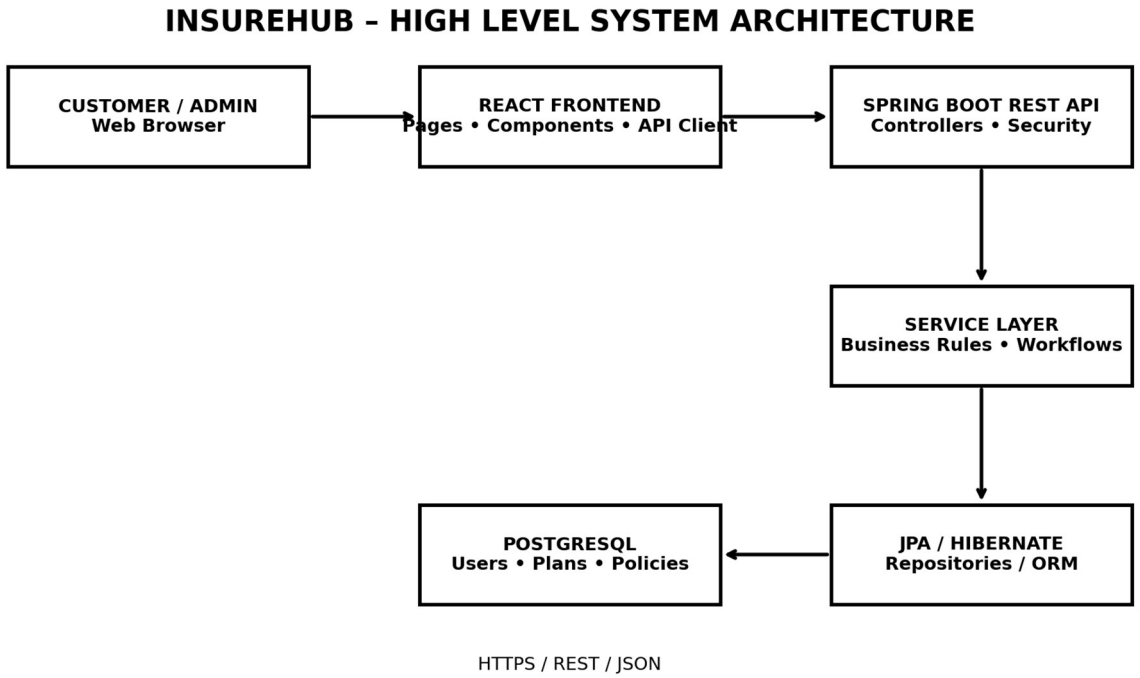


Figure 1 — Complete system at a glance

TEAM GUIDE • BUILD THIS TOGETHER • KEEP THE CODE MODULAR

1. What We Are Building

InsureHub is a role-based insurance policy management website. A customer can register, log in, browse insurance plans, calculate an estimated premium, purchase a policy, view policy details, renew an eligible policy, and request cancellation. An administrator can manage plans, inspect policies, and approve or reject cancellation requests.

The key idea

CRUD is only one part of the project. The main value comes from business workflows: premium calculation, policy state changes, renewal rules, cancellation approval, audit history, authentication, authorization and validation.

Project goals

- Build a real end-to-end web application with separate frontend, backend and database layers.
- Expose a clean REST API that the React frontend consumes.
- Use JWT authentication and role-based authorization.
- Implement meaningful insurance business rules rather than only create/read/update/delete.
- Maintain policy history instead of deleting important records.
- Document and test the API with Swagger/OpenAPI, Postman and automated tests.
- Keep the GitHub repository easy for four members to collaborate on.

What each member should understand

Member	Primary work	Must understand
Member 1 – Backend	Spring Boot, JPA, APIs, security, business logic	Database + API contract + policy lifecycle
Member 2 – Frontend	React pages, components, API integration, UI	API request/response + auth flow
Member 3 – Report	Documentation, diagrams, test evidence, screenshots	Complete feature workflow
Member 4 – PPT/Demo	Presentation, architecture, demo script, results	Why design choices were made

2. System Architecture

The application follows a simple three-tier architecture. Each layer has a clear responsibility.

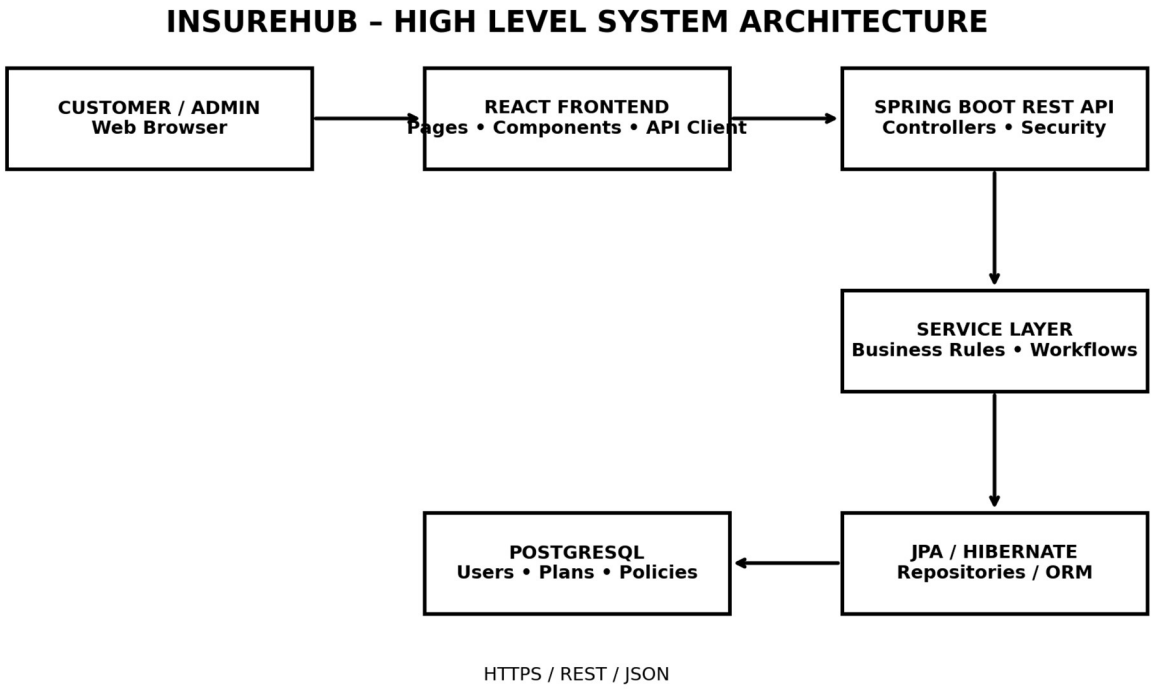


Figure 2 — High-level architecture

How a normal request travels

1. The user clicks a button in the React frontend.
2. React sends an HTTP request to the Spring Boot REST API.
3. Spring Security checks the JWT and required role.
4. The Controller receives and validates the request.
5. The Service layer applies business rules.
6. The Repository uses JPA/Hibernate to access PostgreSQL.
7. The backend returns a JSON response to React.
8. React updates the screen.

Example: Renew Policy

```
React:  POST /api/policies/INS-10231/renew
        ↓
PolicyController
        ↓
RenewalService
        ↓
Check policy status + renewal eligibility
        ↓
Calculate new premium
        ↓
Update expiry + create history entry
        ↓
PolicyRepository → PostgreSQL
        ↓
PolicyResponse JSON → React
```

Rule of thumb

Controllers should be thin. Put actual insurance rules in services. Repositories should focus on database access.

3. Backend Architecture – Easy Version

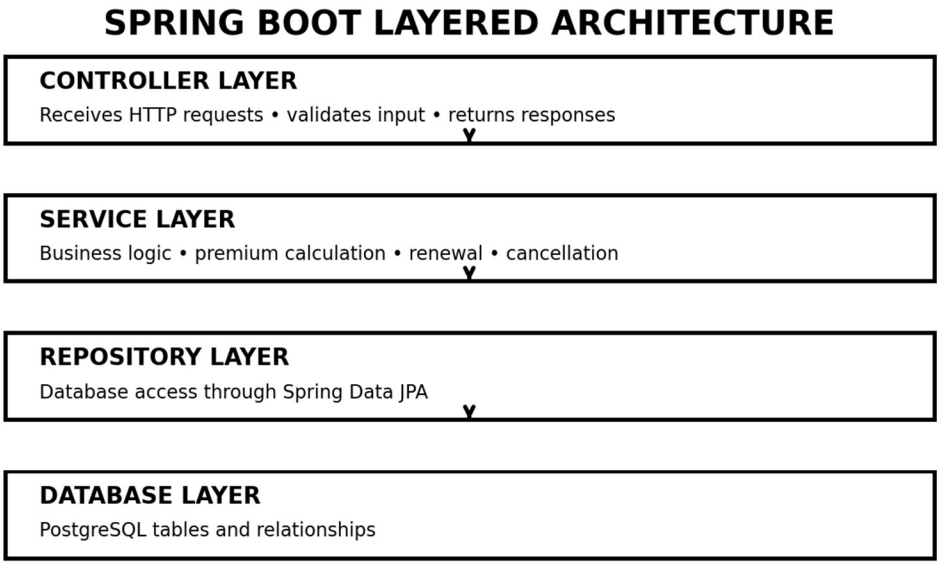


Figure 3 — Spring Boot layered architecture

Layer	What it does	Example classes
Controller	Receives HTTP requests and sends HTTP responses	PolicyController, AuthController
Service	Contains business logic and workflows	PolicyService, RenewalService
Repository	Reads/writes database records	PolicyRepository
Entity	Maps Java objects to database tables	Policy, User, InsurancePlan
DTO	Controls what comes into/out of the API	CreatePolicyRequest, PolicyResponse
Security	Authentication and authorization	JwtFilter, SecurityConfig
Exception	Consistent error responses	GlobalExceptionHandler

Recommended backend package structure

```
backend/
└── src/main/java/com/insurehub/
    ├── controller/
    ├── service/
    ├── repository/
    ├── entity/
    ├── dto/
    ├── security/
    ├── exception/
    ├── config/
    └── util/
```

Why this structure?

If the frontend member changes a screen, the database code should not need to be rewritten. If the premium formula changes, it should mostly affect PremiumService rather than a controller. Separation makes debugging and teamwork easier.

4. Functional Features

Customer features

- Register and log in.
- View available insurance plans.
- Open plan details.
- Calculate an estimated premium.
- Purchase a policy.
- View all own policies.
- Open a policy and see coverage, premium, dates and status.
- Renew an eligible policy.
- Request cancellation.
- View policy history.

Admin features

- Log in as administrator.
- Create/update/deactivate insurance plans.
- View customers and policies.
- View pending cancellation requests.
- Approve/reject cancellation requests.
- View policy history and operational statistics.

Optional features – only after the core is finished

- Payment simulation with a fake transaction record.
- Email-style renewal reminder simulation.
- Claims module.
- Search/filter/sort policies.
- Dashboard charts.
- Export policy data.

Scope control
Do not start with claims, real payment gateways or email integration. Finish authentication → plans → premium → policy → renewal → cancellation → history first.

5. Main Business Workflow – Policy Lifecycle

The policy lifecycle is the feature that turns this project from a basic CRUD system into a workflow-based application.

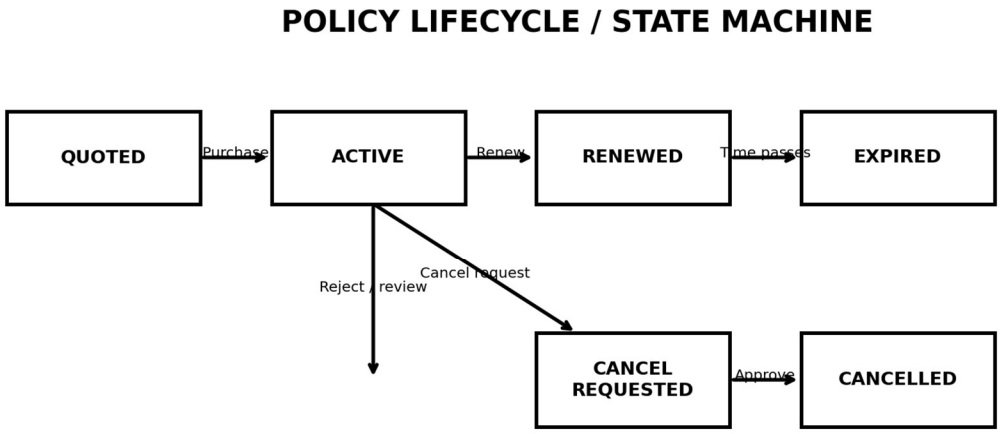


Figure 4 — Policy lifecycle / state machine

Status	Meaning	Typical next action
QUOTED	Premium has been calculated but policy is not active	Purchase
ACTIVE	Policy is currently valid	Renew or request cancellation
RENEWED	Renewal operation has been completed	Continue as active policy
EXPIRED	End date has passed	Renew if business rules allow
CANCEL REQUESTED	Customer asked to cancel; admin review required	Approve / reject
CANCELLED	Policy has been cancelled	View history only

Important business rules

- A cancelled policy must not be renewed.
- A policy should not be physically deleted when cancellation occurs.
- Only the policy owner can operate on their own customer policy.
- Only an authorized admin can approve or reject cancellation requests.
- Renewal must create a history record.
- Cancellation request and decision must create history records.
- Policy number must be unique.
- Dates and coverage must satisfy validation rules.

6. Premium Calculation – Business Logic

The premium calculator is deliberately included so the application performs meaningful computation instead of simply storing form data.

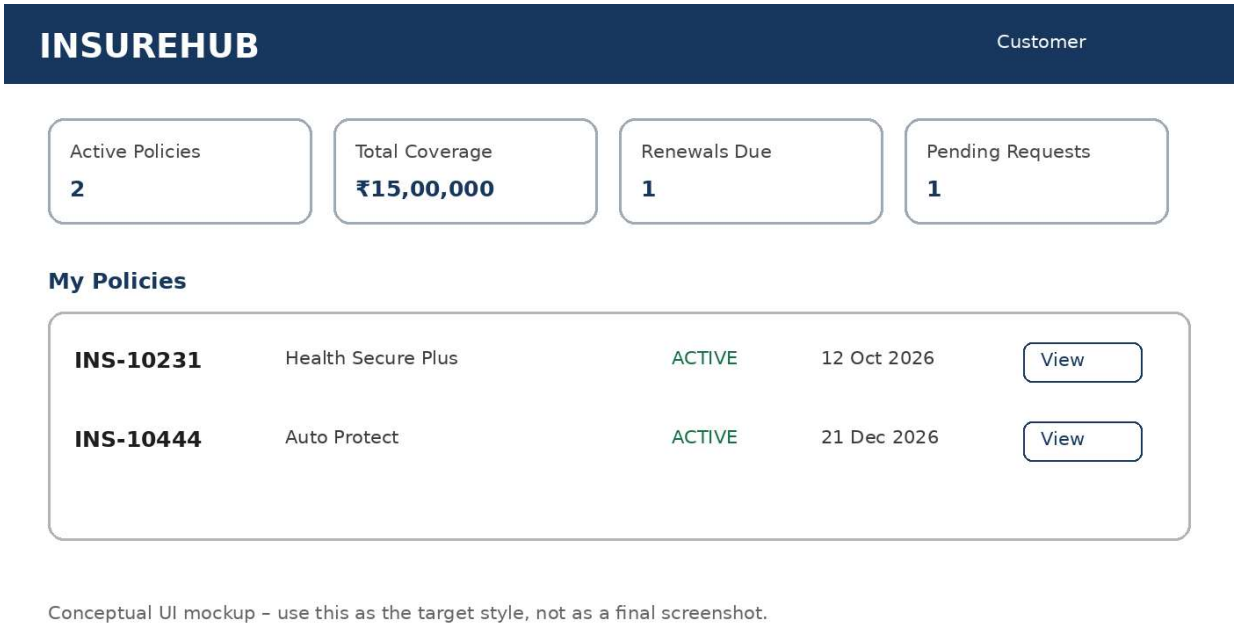


Figure 5 — Conceptual customer dashboard mockup

Suggested formula

```
finalPremium = basePremium
             + ageFactor
             + coverageFactor
             + riskFactor
             - discount
```

Input	Example	Purpose
Plan	Health Secure Plus	Determines base premium
Age	25	Can affect risk/age factor
Coverage	₹5,00,000	Higher coverage can increase premium
Duration	1 year	Changes policy period
Risk category	LOW / MEDIUM / HIGH	Optional simple business rule
Discount	0–10%	Optional controlled discount

API example

```
POST /api/quotes/calculate

Request
{
  "planId": 2,
  "age": 25,
  "coverageAmount": 500000,
  "durationYears": 1
}

Response
{
  "basePremium": 10000,
```

```
  "ageFactor": 500,
  "coverageFactor": 2000,
  "riskFactor": 500,
  "discount": 0,
  "finalPremium": 13000
}
```

Important

The exact formula is a project-defined simulation, not a real insurer's underwriting formula. State this clearly in the report.

7. Database Design

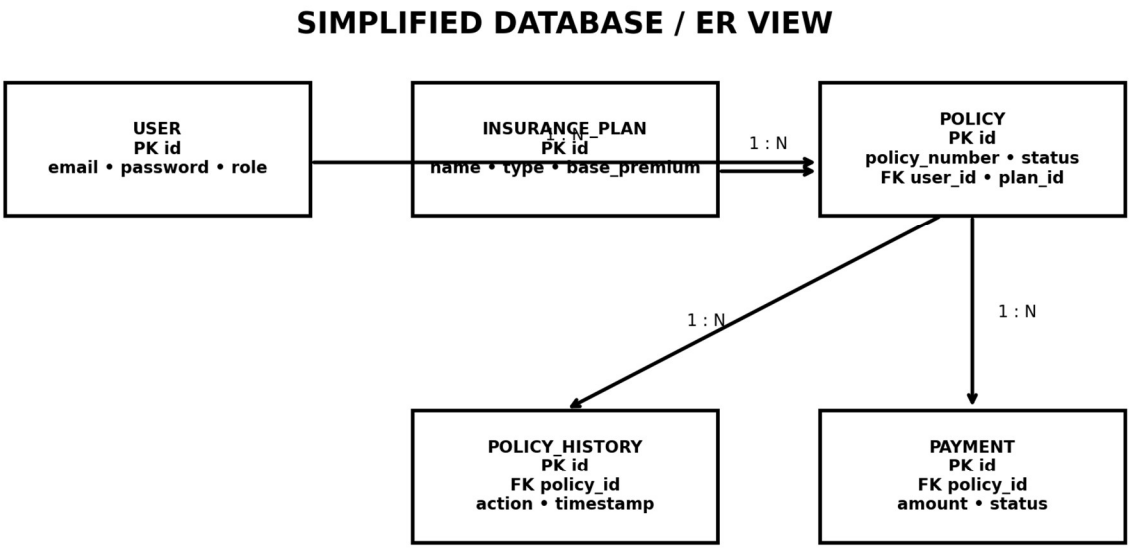


Figure 6 — Simplified ER/database view

Core tables

Table	Important fields	Relationship
USER	id, name, email, password_hash, phone, role, created_at	One user → many policies
INSURANCE_PLAN	id, name, type, description, coverage_amount, base_premium, duration, status	One plan → many policies
POLICY	id, policy_number, user_id, plan_id, premium, dates, status	Belongs to user and plan
POLICY_HISTORY	id, policy_id, action, description, performed_by, timestamp	Many history records → one policy
PAYMENT	id, policy_id, amount, transaction_id, status, payment_date	Many payments → one policy

Relationships

- USER 1 → N POLICY
- INSURANCE_PLAN 1 → N POLICY
- POLICY 1 → N POLICY_HISTORY
- POLICY 1 → N PAYMENT

Do not store passwords as plain text

The User table should contain a password hash produced by a secure password encoder. The frontend should never receive the stored password hash.

8. API Contract – The Bridge Between Frontend and Backend

Before both technical members code heavily, agree on endpoint names, request JSON, response JSON, status codes and authentication requirements. This prevents the frontend and backend from blocking each other.

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Create customer account	Public
POST	/api/auth/login	Authenticate and return JWT	Public
GET	/api/plans	List active insurance plans	Public/Customer
GET	/api/plans/{id}	Get plan details	Public/Customer
POST	/api/plans	Create plan	ADMIN
PUT	/api/plans/{id}	Update plan	ADMIN
POST	/api/quotes/calculate	Calculate premium	Customer
POST	/api/policies	Create/purchase policy	Customer
GET	/api/policies/my	Customer's policies	Customer
GET	/api/policies/{id}	Policy details	Owner/Admin
POST	/api/policies/{id}/renew	Renew policy	Owner
POST	/api/policies/{id}/cancel	Request cancellation	Owner
GET	/api/policies/{id}/history	Policy audit trail	Owner/Admin
GET	/api/admin/cancellation-requests	Pending requests	ADMIN
PUT	/api/admin/cancellation-requests/{id}/approve	Approve cancellation	ADMIN
PUT	/api/admin/cancellation-requests/{id}/reject	Reject cancellation	ADMIN

Example: create policy

```
POST /api/policies
Authorization: Bearer <JWT>

{
  "planId": 2,
  "coverageAmount": 500000,
  "durationYears": 1
}
```

Example response

```
201 Created
{
  "policyNumber": "INS-2026-000182",
  "planName": "Health Secure Plus",
  "premium": 13000,
  "startDate": "2026-09-05",
  "expiryDate": "2027-09-04",
  "status": "ACTIVE"
}
```

9. Authentication & Authorization

Use Spring Security with JWT-based bearer authentication. The login endpoint returns a token. The frontend stores the token in an appropriate client-side mechanism and sends it in the Authorization header for protected API calls.

```
Login
POST /api/auth/login
        ↓
Validate email + password
        ↓
Create JWT containing user identity/role
        ↓
React receives token
        ↓
Authorization: Bearer <token>
        ↓
Spring Security validates token
        ↓
Controller is allowed/blocked
```

Roles

Role	Can do
CUSTOMER	Browse plans, calculate premium, purchase/view/renew/cancel own policies
ADMIN	Manage plans, inspect users/policies, approve/reject cancellation requests
AGENT (optional)	Assist with customer policy operations if the team has time

Security boundary

Never trust a userId sent by the frontend to decide ownership. The backend should derive the authenticated user from the security context/JWT and then check ownership.

Useful HTTP status codes

Code	Meaning
200	Successful request
201	Resource created
400	Invalid request/business rule
401	Not authenticated / invalid token
403	Authenticated but not authorized
404	Resource not found
409	Conflict, e.g. invalid state or duplicate policy number
500	Unexpected server error

10. Validation & Exception Handling

Validation examples

- Email must be valid.
- Required fields cannot be empty.
- Coverage amount must be positive and within the selected plan's limits.
- Duration must be within the plan's allowed duration.
- A cancelled policy cannot be renewed.
- Only the owner/admin can view protected policy information.
- Duplicate policy numbers must be rejected.
- Cancellation should only be requested for valid policy states.

Consistent error response

```
{
  "timestamp": "2026-09-05T15:30:00",
  "status": 409,
  "error": "INVALID_POLICY_STATE",
  "message": "Policy INS-10231 cannot be renewed because it is cancelled."
}
```

Suggested exception classes

- PolicyNotFoundException
- PlanNotFoundException
- UserNotFoundException
- InvalidPolicyStateException
- UnauthorizedPolicyAccessException
- DuplicatePolicyException

Why this matters in viva

Instead of saying 'Spring handles the error', explain that GlobalExceptionHandler converts predictable application failures into clean, consistent HTTP responses for the frontend.

11. Frontend Structure

React is component-based, so keep reusable UI pieces separate from pages and API logic. The frontend should never contain database logic; it calls the backend API.

```
frontend/
└── src/
    ├── components/
    │   ├── Navbar.jsx
    │   ├── PolicyCard.jsx
    │   ├── PlanCard.jsx
    │   └── ProtectedRoute.jsx
    ├── pages/
    │   ├── Login.jsx
    │   ├── Register.jsx
    │   ├── Dashboard.jsx
    │   ├── Plans.jsx
    │   ├── PremiumCalculator.jsx
    │   ├── MyPolicies.jsx
    │   ├── PolicyDetails.jsx
    │   ├── RenewPolicy.jsx
    │   ├── CancelPolicy.jsx
    │   └── AdminDashboard.jsx
    ├── services/
    │   └── api.js
    ├── context/
    │   └── AuthContext.jsx
    ├── hooks/
    ├── utils/
    └── App.jsx
```

Suggested screens

Screen	Purpose
Login/Register	Authentication
Customer Dashboard	Summary of policies and upcoming actions
Plans	Browse insurance plans
Plan Details	Coverage, premium and terms
Premium Calculator	Estimate premium
Purchase Policy	Confirm details and create policy
My Policies	List customer policies
Policy Details	Show full policy + actions
Renew Policy	Show renewal calculation and confirmation
Cancel Policy	Submit cancellation request
Policy History	Show audit trail
Admin Dashboard	Operational summary
Manage Plans	Admin CRUD for insurance plans
Cancellation Requests	Admin approve/reject workflow

12. GitHub Repository & Team Workflow

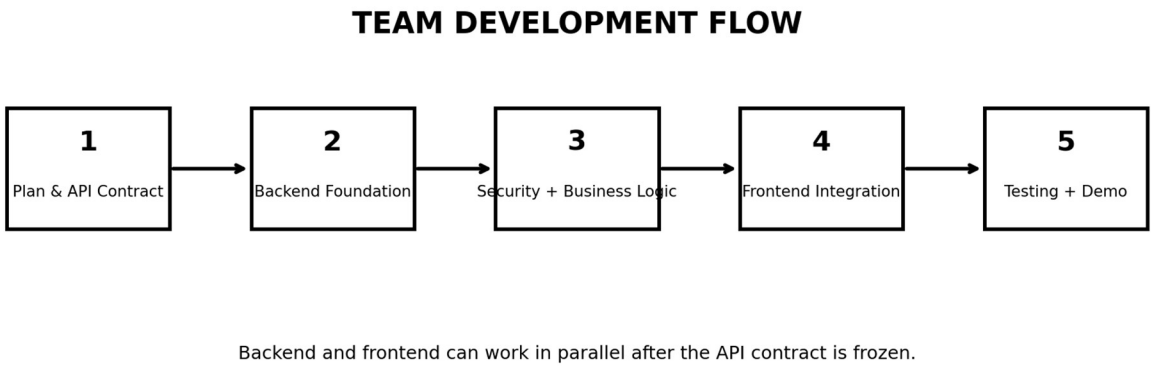


Figure 7 — Recommended development flow

```
insurance-policy-management/
├── backend/
├── frontend/
├── docs/
│   ├── architecture/
│   ├── database/
│   ├── api/
│   ├── screenshots/
│   └── project-report/
├── README.md
└── .gitignore
```

Recommended branches

- main – stable integrated version
- backend-dev – backend member
- frontend-dev – frontend member
- docs-dev – report/PPT members (or separate branches if preferred)

Commit examples

```
feat: add policy creation API
feat: implement JWT login
feat: add renewal workflow
feat: add customer dashboard
docs: add ER diagram
test: add policy renewal tests
```

The most important collaboration file

Create docs/api/API_CONTRACT.md before major frontend/backend development. It should define every endpoint, request JSON, response JSON, authentication requirement and common errors.

13. Team Task Breakdown

Member	Week/Phase	Tasks	Deliverable
Backend	Phase 1	Project setup, DB connection, entities	Spring Boot skeleton + schema
Backend	Phase 2	Repositories, services, controllers	Plan + policy APIs
Backend	Phase 3	JWT, roles, validation, exceptions	Secure API
Backend	Phase 4	Renewal, cancellation, history	Complete workflow
Frontend	Phase 1	Wireframes + routing	Page skeleton
Frontend	Phase 2	Login, plans, dashboard	Core UI
Frontend	Phase 3	Policy purchase, renew, cancel	Workflow UI
Frontend	Phase 4	Admin UI + polish	Integrated frontend
Report	Throughout	Requirements, diagrams, screenshots	Report material
PPT/Demo	Throughout	Architecture, feature slides, demo script	Presentation-ready material

Parallel work rule

Once the API contract is agreed, backend and frontend can work in parallel. The frontend member can use mocked JSON responses while backend endpoints are being developed.

Do not block each other

Frontend should not wait for the backend to be 100% finished. Build screens against the agreed API contract and replace mocks with real Axios calls during integration.

14. Testing Plan

Manual/API test cases

Test	Expected result
Register valid customer	201 Created
Register duplicate email	400/409 with clear error
Login valid credentials	JWT returned
Login wrong password	401
Customer reads own policies	200
Customer reads another user's policy	403/404 according to design
Calculate valid premium	200 + correct calculation
Create policy with invalid plan	400/404
Renew active policy	200 + new expiry/history
Renew cancelled policy	409
Request cancellation	200/202 + history
Admin approves cancellation	200 + status CANCELLED
Non-admin approves cancellation	403
Unknown policy ID	404

Automated tests

- PremiumServiceTest – verify formula and edge cases.
- PolicyServiceTest – create policy and validation.
- RenewalServiceTest – active/expired/cancelled cases.
- CancellationServiceTest – request and approval rules.
- Controller/API tests – status codes and validation responses.

Testing evidence for report

- Swagger screenshots
- Postman request/response screenshots
- JUnit test results
- Database screenshots showing policy history
- Frontend screenshots of successful workflows

15. Final Demonstration Script

Do not demo the project by randomly clicking pages. Tell a short story and demonstrate one complete business journey.

9. Open the application and show the landing/login screen.
10. Register a customer or use a prepared demo customer.
11. Log in and show the customer dashboard.
12. Open insurance plans.
13. Select a plan and calculate premium.
14. Purchase the policy.
15. Open My Policies and show ACTIVE status.
16. Open policy history and show the CREATED/ACTIVATED entry.
17. Renew the policy and show the changed expiry/history.
18. Request cancellation.
19. Log out and log in as ADMIN.
20. Open cancellation requests and approve the request.
21. Open the policy again and show CANCELLED status plus complete history.
22. Open Swagger and show the REST API documentation.
23. Optionally show a failed request to demonstrate validation/security.

Best viva sentence

"The project uses CRUD for persistent resources, but the core functionality is workflow-based: premium calculation, policy state transitions, renewal, cancellation approval, audit history and role-based authorization."

16. Viva / Presentation Talking Points

Question	Short answer
Why Spring Boot?	It provides a clean framework for REST APIs, dependency injection, validation and integration with JPA/security.
Why REST API?	It separates frontend from backend and lets different clients consume the same services.
Why JPA/Hibernate?	It maps Java entities to relational database tables and simplifies persistence.
Why JWT?	It provides bearer-token authentication for protected API requests.
Why roles?	Customers and admins must have different permissions.
Why not delete cancelled policies?	Insurance systems need history/auditability; cancellation is a business state, not deletion.
Where is business logic?	Primarily in services such as PremiumService, RenewalService and CancellationService.
How is it more than CRUD?	Premium calculation, state machine, workflows, authorization, audit history and validation.
How does frontend talk to backend?	Through HTTP REST endpoints returning JSON.
How do you test APIs?	Swagger/Postman for API testing plus JUnit/Mockito for automated tests.

17. Complete Project Folder Blueprint

```
insurance-policy-management/
│
├── backend/
│   ├── src/main/java/com/insurehub/
│   │   ├── controller/
│   │   │   ├── AuthController.java
│   │   │   ├── PlanController.java
│   │   │   ├── PolicyController.java
│   │   │   └── AdminController.java
│   │   ├── service/
│   │   │   ├── AuthService.java
│   │   │   ├── PremiumService.java
│   │   │   ├── PolicyService.java
│   │   │   ├── RenewalService.java
│   │   │   └── CancellationService.java
│   │   ├── repository/
│   │   ├── entity/
│   │   ├── dto/
│   │   ├── security/
│   │   ├── exception/
│   │   └── config/
│   ├── src/test/
│   └── pom.xml
│
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── context/
│   │   ├── hooks/
│   │   └── utils/
│   └── package.json
│
├── docs/
│   ├── architecture/
│   ├── database/
│   ├── api/
│   ├── screenshots/
│   └── project-report/
│
└── README.md
```

Suggested README sections

- Project overview
- Features
- Architecture
- Technology stack
- Folder structure
- How to run backend
- How to run frontend
- Database setup
- Environment variables
- API/Swagger link
- Demo credentials
- Team members and responsibilities

18. Implementation Roadmap

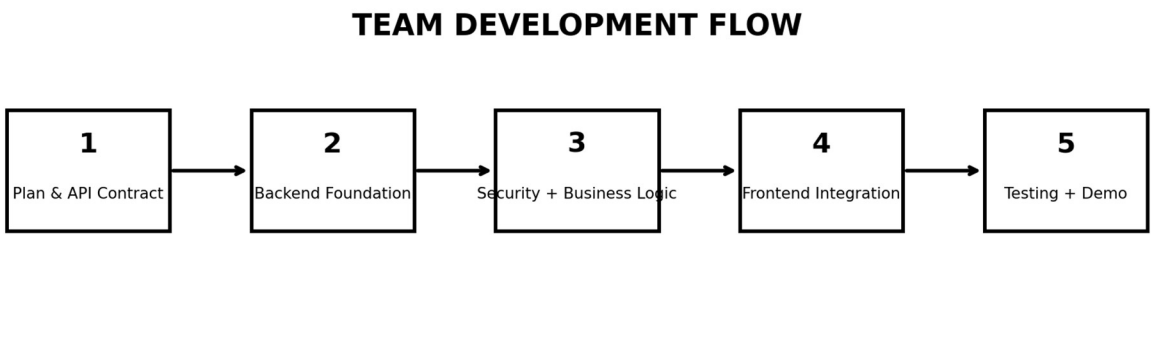


Figure 8 — Build in this order

Phase	Goal	Definition of done
1. Planning	Freeze requirements and API contract	ERD + endpoints + screen list agreed
2. Backend foundation	DB + entities + repositories	Application connects to PostgreSQL
3. Core APIs	Plans + auth + policies	Basic end-to-end API works
4. Security	JWT + roles	Protected endpoints behave correctly
5. Business workflows	Premium + renew + cancel + history	Complete policy lifecycle works
6. Frontend	Customer + admin UI	All major workflows work through UI
7. Integration	Connect real frontend/backend	No major mock data remains
8. Testing	Unit + API + manual	Test evidence collected
9. Documentation	Report + PPT + screenshots	Submission package complete
10. Final demo	One clean story	Team can explain architecture and code

Recommended stopping point

If time becomes short, stop adding features after the complete customer/admin policy lifecycle. A polished core project is better than five unfinished modules.

19. Technology References

The implementation choices in this guide align with the official documentation for React and Spring technologies. These links are included mainly so the team can learn the framework concepts from primary sources.

React Quick Start — Official React guide covering components, JSX, state, events and lists.

https://react.dev/learn

React Installation / App Setup — Official guidance for starting React projects; Create React App is deprecated.

https://react.dev/learn/installation

Spring REST Guide — Official Spring tutorial for building REST services with Spring Boot.

https://spring.io/guides/tutorials/rest/

Spring Data REST / JPA — Spring guide showing relational JPA data exposed through REST.

https://spring.io/guides/gs/accessing-data-rest/

Spring Security — Spring Boot security reference, including method-level security.

https://docs.spring.io/spring-boot/3.5/reference/web/spring-security.html

Spring Security JWT Resource Server — Official Spring Security reference for bearer tokens including JWT.

https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/index.html

springdoc-openapi — OpenAPI/Swagger integration for Spring Boot.

https://github.com/springdoc/springdoc-openapi

Final checklist

- ☐ GitHub repository created with backend/, frontend/, docs/
- ☐ ER diagram finalized
- ☐ API contract finalized
- ☐ Backend connects to PostgreSQL
- ☐ JWT login works
- ☐ Customer/admin roles work
- ☐ Premium calculation works
- ☐ Policy creation works
- ☐ Renewal works
- ☐ Cancellation request + admin approval works
- ☐ Policy history works
- ☐ Frontend integrated
- ☐ Swagger works
- ☐ Tests and screenshots collected
- ☐ Report and PPT completed
- ☐ Final demo rehearsed